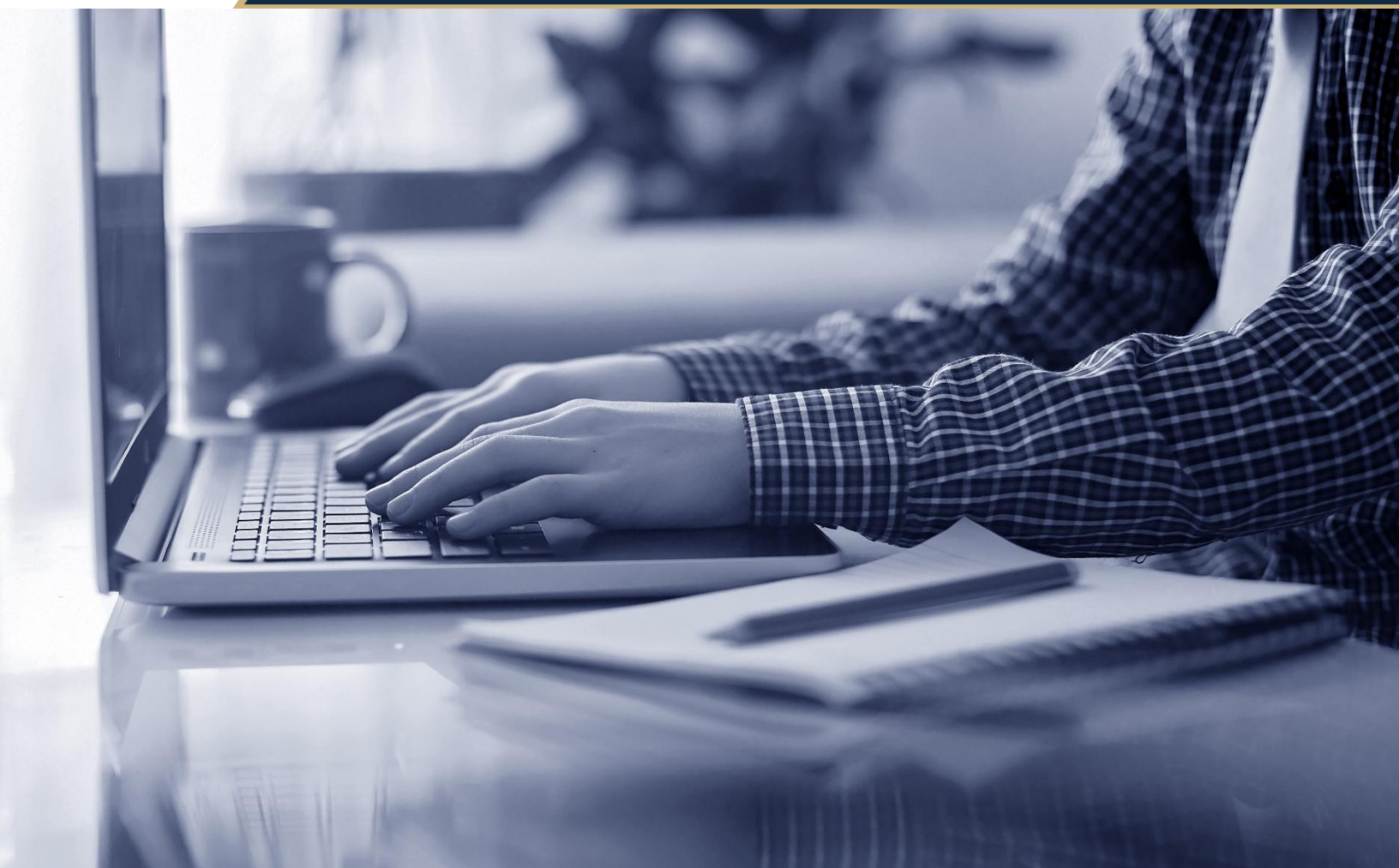




PROGRAMACIÓN PARA LA CIENCIA DE DATOS

Código: CD106ICDA

Apunte N° 2

Algoritmos para procesamiento y transformación de datos

Introducción

El presente documento tiene como finalidad guiar al y la estudiante en el desarrollo de habilidades para programar algoritmos orientados al procesamiento y transformación de datos, fortaleciendo la capacidad de estructurar procesos lógicos y eficientes que preparen la información para su posterior análisis dentro del campo de la ciencia de datos. Mientras que en la Unidad 1 se abordaron los fundamentos esenciales de la programación, tipos de datos, estructuras de control, funciones, lectura y organización de datos mediante librerías como *pandas* y *NumPy*, en esta Unidad 2 se avanza hacia un nivel superior de aplicación, centrado en cómo diseñar, implementar y validar algoritmos capaces de limpiar, transformar y combinar datos de forma integral y automatizada.

Esta unidad profundiza en la planificación algorítmica a través del uso de pseudocódigo y diagramas de flujo, herramientas que permiten estructurar el pensamiento lógico antes de codificar. A partir de ello, se estudiarán las principales técnicas de preprocesamiento de datos, incluyendo la detección y tratamiento de valores atípicos (*outliers*), el manejo de datos faltantes y la limpieza profunda de registros mediante la estandarización de formatos y la eliminación de duplicados. Estos procedimientos son esenciales para garantizar la calidad y consistencia de los datos, asegurando que los resultados obtenidos en fases posteriores sean confiables y reproducibles.

Posteriormente, se abordará la transformación y enriquecimiento de datos, un conjunto de operaciones destinadas a adecuar los datos a distintos propósitos analíticos. En este bloque se revisarán procedimientos como la normalización o escalamiento de valores numéricos, la conversión de tipos de datos (por ejemplo, el parseo de fechas o la categorización de texto), y la codificación de variables categóricas que permiten representar información cualitativa en formato numérico. Estas transformaciones serán implementadas en Python mediante ejemplos prácticos, adaptados a contextos chilenos y de fácil interpretación, que permitirán conectar la teoría con situaciones reales del entorno nacional.

Además, se explorarán los procesos de agregación y combinación de datos, esenciales para la consolidación de información proveniente de distintas fuentes. El uso de funciones de agrupamiento (*groupby*) y de operaciones de fusión (*merge* o *join*) permitirá comprender cómo sintetizar y unir conjuntos de datos de manera coherente, obteniendo resúmenes estadísticos o estructuras unificadas listas para el análisis. A través de estos ejercicios, se busca que el estudiante adquiera una visión más integral del ciclo de tratamiento de datos, comprendiendo cómo cada paso contribuye a obtener una base depurada y enriquecida.

En paralelo, se enfatizarán las buenas prácticas de programación estructurada y modular, fomentando la escritura de código claro, legible y reutilizable mediante funciones y módulos bien definidos. Se promoverá el uso de comentarios, documentación interna y convenciones de estilo que favorezcan la mantenibilidad del código. Asimismo, se introducirá la diferenciación entre datos estructurados y no estructurados, analizando las consideraciones básicas para su tratamiento, con especial atención a los datos tabulares, textos y estructuras semiestructuradas como JSON.

Un aspecto clave de esta unidad es la validación de algoritmos de transformación, donde se enseñará a verificar la corrección de los resultados mediante pruebas, revisiones de consistencia y comparación de totales antes y después del procesamiento. Finalmente, se incorporarán nociones de eficiencia y complejidad computacional, destacando la importancia de optimizar el uso del tiempo y los recursos en el procesamiento de volúmenes moderados de datos, eligiendo las estructuras y métodos adecuados para cada tarea.

A lo largo de este apunte activador, se presentarán ejemplos funcionales en Python, gráficos ilustrativos y ejercicios guiados que permitirán aplicar los conceptos de manera práctica y progresiva. Cada tema se enlazarará con el siguiente, de modo que el estudiante logre construir un flujo de trabajo coherente que va desde el diseño del algoritmo hasta su validación final. Al término de la unidad, el estudiante estará en condiciones de diseñar e implementar algoritmos robustos para el procesamiento y transformación de datos, cumpliendo con los resultados de aprendizaje definidos para el curso.

Éxito en el estudio de este material, que constituye una etapa clave en la formación como programador o programadora en ciencia de datos, fortaleciendo las competencias necesarias para manipular información de manera eficiente, ética y profesional en escenarios reales del ámbito tecnológico y analítico.

Diseño de algoritmos para el procesamiento de datos.

En unidades anteriores aprendimos los fundamentos de la programación (tipos de datos, estructuras de control, etc.). Ahora, damos un paso más: antes de escribir código Python, es fundamental *estructurar el pensamiento lógico* mediante el diseño de algoritmos. Esto implica planificar la secuencia de pasos necesarios para resolver un problema de datos, antes de codificar. Dos herramientas clásicas para este diseño preliminar son el pseudocódigo y los diagramas de flujo.

Pseudocódigo: es una descripción en lenguaje semi-natural (ej. en español, con palabras clave similares a las de programación) que detalla los pasos del algoritmo. No sigue la sintaxis exacta de ningún lenguaje, sino que sirve para explicar la lógica de forma fácil de leer. Por ejemplo, podríamos escribir en pseudocódigo: “Inicio -> Leer datos -> Si el dato es mayor a 100, marcar como ‘alto’ -> Fin”. El pseudocódigo nos obliga a pensar en la solución paso a paso, separando la lógica del problema de detalles de sintaxis del lenguaje.

Diagrama de flujo: representa gráficamente un algoritmo, usando símbolos estandarizados para cada tipo de acción (inicio/fin, proceso, decisión, entrada/salida, etc.). Los pasos se dibujan en cajas conectadas por flechas que indican el flujo de ejecución. Un diagrama de flujo bien hecho permite *visualizar* la ruta que seguirá el programa según las decisiones lógicas, lo que ayuda a identificar errores lógicos o casos no contemplados, incluso antes de programar.

¿Por qué usar pseudocódigo y diagramas? Porque ayudan a estructurar el pensamiento. Es común atascarse codificando si la solución no está clara; al diseñar primero el algoritmo de forma abstracta, nos aseguramos de que la lógica sea sólida. Además, estas herramientas facilitan la comunicación: un diagrama o pseudocódigo bien comentado puede ser entendido por otros (incluso por personas que no dominan Python), alineando al equipo en cómo se abordará el procesamiento de datos.

A continuación, veamos un ejemplo sencillo. Supongamos que tenemos una lista de números (por ejemplo, datos numéricos crudos) y queremos construir un algoritmo que cuente cuántos valores positivos, negativos y cero hay en la lista. Antes de codificar, escribiremos el pseudocódigo y dibujaremos el diagrama de flujo:

```
INICIO
establecer contador_positivos = 0
establecer contador_negativos = 0
establecer contador_ceros = 0

PARA CADA número EN la_lista:
    SI número < 0 ENTONCES
        incrementar contador_negativos
    SI NO SI número == 0 ENTONCES
        incrementar contador_ceros
    SI NO
        incrementar contador_positivos

mostrar ("Negativos:", contador_negativos,
        "Ceros:", contador_ceros,
        "Positivos:", contador_positivos)
FIN
```

En el pseudocódigo anterior describimos la lógica: inicializar contadores, iterar por cada elemento y usar estructuras condicionales para clasificar el número como negativo, cero o positivo, incrementando el contador correspondiente, y al final mostrar los resultados. Ahora veamos el mismo algoritmo en un diagrama de flujo:

Diagrama de flujo – Algoritmo que recorre una lista de números y cuenta cuántos son positivos, negativos y ceros. Cada figura representa un paso: el rombo indica una decisión condicional (evaluar el signo del número), los rectángulos son procesos (incrementar contadores) y el óvalo es el inicio/fin. Las flechas dirigen el flujo según las condiciones.

Como se observa en la descripción anterior, partimos del inicio, luego por cada número verificamos la condición en un símbolo de decisión (“¿número < 0?”, “¿número = 0?”) y seguimos la rama correspondiente: sumar a negativos, a ceros, o a positivos. Al final, el flujo llega al proceso de mostrar resultados y termina. Este diagrama de flujo corresponde paso a paso con el pseudocódigo diseñado. Notemos que estructurar el algoritmo antes de codificar nos permite detectar detalles lógicos: por ejemplo, nos aseguramos de cubrir todas las posibilidades de signo del número (negativo, cero, positivo) mediante un SI/SI NO SI/SI NO encadenado. Si olvidáramos una condición, el diagrama tendría una rama incompleta, evidenciando el error lógicamente antes de implementarlo en código.

En resumen, el diseño de algoritmos con pseudocódigo y diagramas de flujo es un hábito clave para cualquier *científico de datos programador*. Antes de sumergirnos en librerías como pandas o en escribir funciones complejas, debemos tener clara la hoja de ruta algorítmica para procesar nuestros datos. Una vez el algoritmo está diseñado y revisado, estamos listos para pasar al siguiente paso: preparar los datos (preprocesamiento) según ese plan.

2. Técnicas de preprocesamiento de datos

Una vez definido qué hará nuestro algoritmo, el primer gran bloque de operaciones suele ser el preprocesamiento de datos. El preprocesamiento consiste en *limpiar y preparar los datos brutos* antes de cualquier análisis profundo, asegurando su calidad y coherencia. En esta sección abordaremos algunas técnicas esenciales de preprocesamiento: tratamiento de valores atípicos (outliers), manejo de valores faltantes, y la limpieza profunda (formato consistente, eliminar duplicados, etc.). Cada subtema se conecta con el anterior: tras diseñar la lógica general, comenzamos por sanear los datos de entrada.

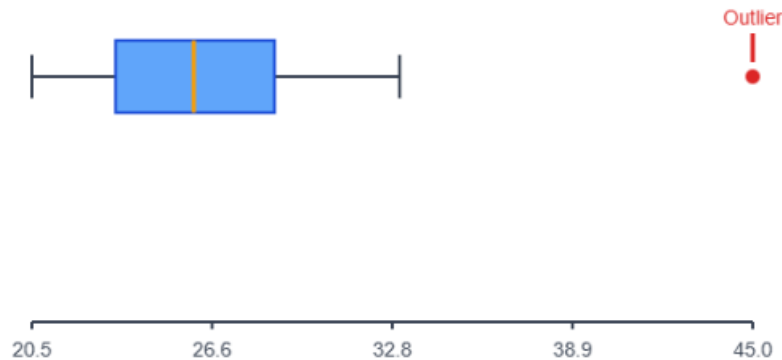
Tratamiento de valores atípicos (outliers)

Los valores atípicos son datos que se alejan significativamente del resto de observaciones. Pueden distorsionar análisis estadísticos y modelos si no se manejan apropiadamente. En contextos chilenos, pensemos en ejemplos: si analizamos la inflación anual de Chile, la mayoría de los valores en las últimas décadas están en torno al 3%; pero el año 2022 sobresalió con cerca de 12,8% (la inflación más alta en 30 años). Ese año con ~12% de inflación sería un outlier respecto a la tendencia histórica normal del 3%. Otro ejemplo: en un conjunto de datos de Índice de Masa Corporal (IMC) de una población, donde la mayoría oscila entre digamos 20 y 30, un valor de IMC = 60 sería extremadamente inusual y sospechoso (posiblemente un error de registro o un caso excepcional).

¿Por qué importan los outliers? Porque pueden sesgar promedios, escalas y visualizaciones. Por ejemplo, si calculamos el ingreso promedio de un grupo e incluimos a un multimillonario, el promedio se disparará y no representará al grupo típico. Por ello, al preprocesar, debemos identificar y decidir cómo manejar estos valores. Las estrategias comunes incluyen: *eliminar* el outlier si se considera error o irrelevante, *winsorizar* o limitar su valor a un percentil máximo (ej. reemplazarlo por el valor del percentil 95), o *tratarlo aparte* (analizar por separado).

Identificación de outliers: Se puede hacer mediante técnicas estadísticas (por ejemplo, valores que caen fuera de 3 desviaciones estándar de la media, o fuera del rango intercuartílico en un *boxplot*). Visualizar los datos ayuda mucho. Por ejemplo, a continuación, vemos un diagrama de caja (boxplot) de una lista de valores de IMC, donde se aprecia un punto atípico muy alto.

Imagen 1: Grafico Bloxplot de valores de IMC



Fuente: Creado con IA

En el gráfico tipo boxplot de valores de IMC. En la caja se concentra el 50% central de los datos (entre Q1 y Q3), la línea central es la mediana (~valor típico), y los “bigotes” suelen extenderse hasta 1.5 veces el rango intercuartílico. El punto aislado por encima es un valor atípico (un IMC mucho mayor que el resto, potencialmente erróneo o extraordinario). Gráficas así nos alertan de outliers rápidamente.

Tratamiento de outliers: Una vez detectados, ¿qué hacemos? Depende del caso y de la causa del outlier. Si sospechamos que es un error de ingreso de datos (ej. un “0” demás que infló un valor 10 veces), quizás lo eliminamos o corregimos. Si el valor es real pero poco común, podríamos excluirlo del cálculo de ciertas métricas para no sesgar, o informar por separado. Para simplificar, veamos cómo eliminar outliers en código con un criterio sencillo: eliminar cualquier valor que supere cierto umbral definido. Por ejemplo, supongamos que decidimos que cualquier IMC mayor a 50 es anómalo y queremos filtrarlo fuera:

```
bmi = [22, 27, 19, 31, 60] # Lista de IMC, donde 60 es sospechosamente alto
# Eliminación de outlier: remover valores > 50
bmi_sin_outliers = [x for x in bmi if x <= 50]
print(bmi_sin_outliers) # Salida esperada: [22, 27, 19, 31]
```

En este código definimos una lista `bmi` con un outlier (60). Luego usamos una *lista por comprensión* para filtrar solo los valores menores o iguales a 50, almacenando el resultado en `bmi_sin_outliers`. Al imprimir, obtenemos `[22, 27, 19, 31]`, donde el 60 fue removido. En la práctica, podríamos automatizar el umbral usando estadísticos (por ej., descartar $> Q3 + 1.5 \cdot RIQ$), pero el ejemplo ilustra el concepto.

Nota: No siempre conviene eliminar outliers de inmediato; a veces conviene *imputarlos* (sustituirlos por un valor más razonable). Por simplicidad aquí mostramos eliminación, pero siempre debemos justificar la decisión según el contexto de los datos.

Manejo de valores faltantes (datos ausentes)

Otro problema común en datos reales son los valores faltantes (missing values), que aparecen como espacios en blanco, NA, None, NaN, etc., cuando cierta observación no tiene un valor registrado. Por ejemplo, en un dataset de asistencia escolar podría faltar el dato de asistencia de un alumno en cierto día; en una encuesta, un encuestado puede haber omitido responder su edad. Los valores faltantes pueden dificultar el análisis (muchas herramientas no calculan promedios si hay datos faltantes, por ejemplo) y por eso debemos decidir cómo tratarlos en la etapa de preprocesamiento.

Detección: En Python (pandas) los valores faltantes suelen representarse como NaN (Not a Number) o podemos tener None en listas nativas. Primero, conviene *explorar cuántos faltantes hay* en cada columna para entender la magnitud del problema (ej. usando `df.isnull().sum()` en pandas). Una vez detectados, hay dos enfoques principales:

- **Eliminar registros faltantes:** Si una fila tiene datos críticos faltantes, a veces la opción es eliminar esa fila (o eliminar la columna si falta demasiada información en esa variable). Esto reduce la cantidad de datos, pero garantiza que los restantes estén completos. Se hace con cuidado: por ejemplo, si tenemos 10% de filas con un campo faltante, quizás eliminarlas está bien; pero si es 50%, conviene pensar en otra estrategia.
- **Imputar o rellenar los faltantes:** Consiste en *estimar* o completar el valor faltante con algún criterio. Opciones comunes: reemplazar por un valor estadístico (media, mediana o moda de esa variable), usar un valor por defecto ("N/A" para texto, 0 para números en algunos casos), o métodos más avanzados como imputación por modelos predictivos. La imputación evita perder datos, aunque introduce un elemento de inferencia que puede agregar sesgo si no se hace apropiadamente.

Veamos un ejemplo sencillo en Python para ilustrar ambos enfoques. Supongamos que tenemos una lista de números donde uno de los elementos es None (faltante):

```
data = [5, None, 3, 8] # Lista con un valor faltante (None) en la segunda posición
# 1. Eliminar valores faltantes:
datos_sin_nan = [x for x in data if x is not None]
print(datos_sin_nan) # [5, 3, 8]

# 2. Imputar el faltante con la media de los valores presentes:
media = sum([x for x in data if x is not None]) / len([x for x in data if x is not None])
datos_imputados = [x if x is not None else media for x in data]
print([round(x, 2) for x in datos_imputados]) # [5, 5.33, 3, 8]
```

En este código, la lista original `data` contiene `[5, None, 3, 8]`. Primero, removemos el `None` usando nuevamente una comprensión de lista que filtra solo valores no-`None`, obteniendo `[5, 3, 8]`. Luego calculamos la media de los datos conocidos (5, 3 y 8), que es aproximadamente 5.33, y creamos una nueva lista `datos_imputados` donde sustituimos el `None` por dicha media. El resultado impreso es `[5, 5.33, 3, 8]`, donde el segundo elemento faltante fue reemplazado por ~5.33 (redondeado a dos decimales).

Ambas estrategias tienen pros y contras: al eliminar, preservamos la autenticidad de datos restantes, pero sacrificamos cantidad (aquí perdimos un valor, aunque mínimo); al imputar, mantenemos cantidad pero introducimos un valor "inventado" basado en suposiciones (asumimos que el promedio era una buena estimación). La mejor elección depende del contexto y de cuán importantes sean esos registros.

Limpeza profunda de datos (formatos, consistencia y duplicados)

Además de outliers y faltantes, la limpieza de datos incluye una variedad de tareas para asegurar que los datos estén *consistentes y libres de ruido*. Esto abarca:

- **Estandarización de formatos:** Asegurarse de que todos los datos sigan una misma representación. Ejemplos: unificar el formato de fechas (si unos registros usan DD-MM-YYYY y otros YYYY/MM/DD hay que elegir uno y convertir), uniformar unidades (si una columna es “altura” y algunas entradas están en metros y otras en centímetros, convertir todas a una unidad común), o homogenizar texto (por ejemplo, que “Sí” y “sí” y “Si” se representen igual, tal como “sí” en minúsculas). Esto evita duplicidades o confusiones. Por ejemplo, podríamos tener una columna “Región” donde unas filas digan “Metropolitana” y otras “metropolitana” – conviene dejar todo en minúsculas o con Capitalización consistente para tratarlas igual.
- **Eliminación de duplicados:** Datos duplicados ocurren cuando la misma observación aparece repetida. Un caso típico en datos públicos es cuando se combinan fuentes y sin querer se repiten filas, o cuando un formulario fue enviado dos veces. Las duplicidades pueden distorsionar análisis (contar algo doble). Detectarlos es relativamente fácil: en pandas se usa `df.duplicated()` para marcar duplicados. Podemos eliminarlos con `df.drop_duplicates()`. En una lista Python simple, podríamos convertirla a conjunto (`set`) para ver los únicos, aunque esto pierde el orden. A continuación, un ejemplo conceptual:

```
nombres = ["Juan", "Ana", "Juan", "Luis"]
unicos = list(set(nombres))
print(unicos) # Ejemplo de salida: ['Ana', 'Juan', 'Luis']
```

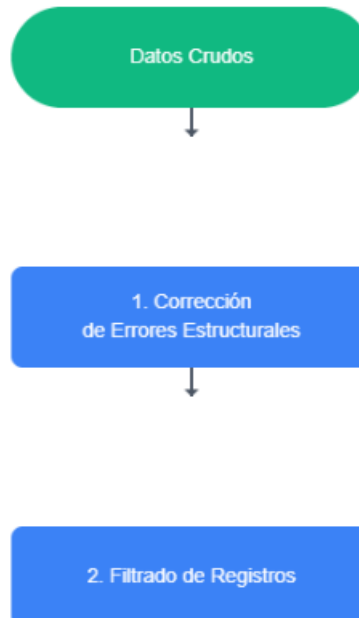
Aquí la lista original tenía “Juan” repetido. Tras convertir a `set` y de vuelta a `list`, obtuvimos cada nombre solo una vez (el orden puede cambiar porque los conjuntos no preservan orden). En pandas, `drop_duplicates` preservaría el primer “Juan” y eliminaría el segundo automáticamente.

- **Corrección de errores tipográficos y categóricos:** A veces “valores atípicos” no numéricos se dan por formatos mal escritos. Ejemplo: en una columna de género donde esperamos “M” o “F”, encontrar “Masculino” escrito completo o “femenino ” con un espacio extra al final, etc. Estas inconsistencias se corrigen reemplazando strings, eliminando espacios, convirtiendo todo a mayúsculas/minúsculas, etc. Ejemplo: `df['Genero'] = df['Genero'].str.strip().str.upper()` eliminaría espacios y pondría todo en mayúsculas (“femenino ” -> “FEMENINO”). Así evitamos tener categorías separadas que en realidad son la misma.
- **Integridad general y reglas de negocio:** Verificar que los datos cumplan reglas lógicas. Por ejemplo, en un dataset de personas, nadie debería tener edad negativa ni 200 años; en una tabla de notas, el promedio no puede exceder el máximo posible. Detectar y corregir estas cosas forma parte de la limpieza profunda.

Todas estas tareas de limpieza suelen realizarse en conjunto, iterando hasta dejar el dataset listo. Podemos visualizar el proceso de limpieza de datos como un flujo que lleva datos *en bruto* hacia datos *consistentes*. El siguiente esquema general ilustra este flujo:

Proceso general de limpieza de datos. Los datos crudos pasan por diversas etapas de limpieza: primero se corrigen errores estructurales (formato, tipos de datos, etiquetas mal escritas), luego se filtran duplicados o registros irrelevantes, se manejan los valores atípicos (outliers) y los valores faltantes, y finalmente se realiza una validación de calidad. El objetivo es obtener datos técnicamente correctos y consistentes listos para análisis.

Imagen 2: Proceso general de limpieza de datos



Fuente: creada con IA

3. Transformación y enriquecimiento de datos

Con datos ya limpios y depurados, el siguiente enfoque es transformarlos y enriquecerlos para facilitar su análisis. Transformar implica cambiar la forma o escala de los datos (por ejemplo, normalizar valores, convertir formatos), y enriquecer implica derivar nueva información a partir de la existente (por ejemplo, extraer categorías de un texto, crear columnas adicionales, etc.). Estas operaciones permiten adaptar los datos a requerimientos específicos de algoritmos o a comprensiones más profundas. En esta sección abordaremos varias transformaciones comunes: normalización de datos numéricos, parseo de fechas, categorización de texto, y codificación de variables categóricas.

Normalización o escalamiento de valores numéricos

En ciencia de datos frecuentemente necesitamos normalizar o escalar valores numéricos, especialmente cuando diferentes variables tienen rangos muy distintos. Por ejemplo, si tenemos la altura de personas (en cm) y su ingreso anual (en pesos), los rangos son totalmente distintos (altura ~150-200, ingreso puede ser en millones). Muchos algoritmos (ej. métodos de aprendizaje automático) funcionan mejor si estos atributos están en escalas comparables. La normalización lineal típica lleva los valores a un rango común, como [0,1] o [-1,1]. Una forma simple es escalamiento min-max: transformar cada valor usando la fórmula $\text{valor_norm} = (\text{valor} - \text{min}) / (\text{max} - \text{min})$, de modo que el mínimo de la serie se convierte 0 y el máximo 1, con los demás proporcionalmente entre 0 y 1.

Imaginemos un ejemplo concreto: supongamos que son puntajes de cierta prueba estandarizada que van de 50 a 100 (sobre 100 puntos), pero queremos normalizarlos de 0 a 1. Tomemos una lista de puntajes [50, 80, 90, 100]. Apliquemos min-max manualmente:

```
valores = [50, 80, 90, 100]
min_v = min(valores) # 50
max_v = max(valores) # 100

normalizados = [(x - min_v) / (max_v - min_v) for x in valores]
print([round(x, 2) for x in normalizados]) # [0.0, 0.60, 0.80, 1.0]
```

Aquí el valor más bajo (50) se transformó en 0.0 y el más alto (100) en 1.0. Un puntaje de 80 quedó en 0.60 (es decir, 60% del camino entre 50 y 100) y un 90 quedó en 0.80. Ahora todos los puntajes están en un mismo rango [0,1].

La normalización facilita comparaciones: por ejemplo, si normalizamos alturas e ingresos por separado, ambos quedarán entre 0 y 1, y se puede ver quién está relativamente más “alto” o “rico” respecto a su grupo. Otra transformación común es la estandarización a media 0 y desviación estándar 1 (restar la media y dividir por la desviación estándar), útil en estadística. En nuestro contexto, normalizamos sobre todo para preparar datos para algoritmos que son sensibles a escalas (muchos modelos de machine learning lo son). Tras normalizar, ningún valor dominará por tener escala mayor.

Parseo de fechas (conversión de textos a formato fecha)

Los datos de fechas y horas son muy valiosos (nos permiten analizar tendencias temporales, calcular duraciones, etc.), pero vienen frecuentemente como *texto* crudo en los datasets (por ejemplo, “31-12-2023” como cadena de caracteres). Para poder explotarlos, debemos *parsearlos* o convertirlos a un tipo de dato de fecha/tiempo apropiado (en Python, un objeto `datetime`). Una vez convertidos, podemos ordenar cronológicamente, extraer componentes (año, mes, día), calcular diferencias (edad = hoy - fecha_nacimiento), etc., con facilidad.

El parseo requiere conocer el formato. En Chile es común ver fechas como DD-MM-AAAA, por ejemplo “31-12-2023” para 31 de diciembre de 2023. Python no adivina el formato automáticamente; debemos indicárselo. Usemos el módulo `datetime` para convertir una cadena en formato día-mes-año:

```
from datetime import datetime
fecha_str = "31-12-2023" # Fecha en formato D-M-A (día-mes-año)
fecha_obj = datetime.strptime(fecha_str, "%d-%m-%Y")
print(fecha_obj.date()) # 2023-12-31
```

En este código, `datetime.strptime` significa “string parse time”, y toma dos argumentos: la cadena de fecha y una *máscara de formato*. `%d` representa día (01-31), `%m` mes (01-12) y `%Y` año con 4 dígitos. Por tanto “%d-%m-%Y” coincide con “31-12-2023”. El resultado `fecha_obj` es un objeto `datetime` que internamente representa ese momento (31 dic 2023 a las 00:00). Al imprimir `fecha_obj.date()` vemos la fecha en formato estandarizado ISO (YYYY-MM-DD). Ahora Python entiende esa variable como fecha; podríamos sumarle días, compararla con otras fechas, etc.

Importante: Al parsear fechas en datasets completos, pandas ofrece la función `pd.to_datetime` que intenta inferir formatos o acepta un formato explícito y convierte una columna entera de golpe. E.g., `df['Fecha'] = pd.to_datetime(df['Fecha'], format="%d-%m-%Y")`. De cualquiera de las formas, este paso es esencial para enriquecer los datos: un campo de fecha bien formateado permite extraer *nuevas características* (por ejemplo, el día de la semana, el trimestre, “fin de semana vs día hábil”, etc.), lo cual añade valor informativo al conjunto de datos.

Categorización de textos

Aquí entramos a un tema más abierto: muchos datos no vienen en números o fechas, sino en texto libre (comentarios, descripciones, respuestas de encuestas abiertas, etc.). *Categorizarlos* significa asignarles alguna etiqueta o clasificación útil. Por ejemplo, podría interesarnos leer comentarios de usuarios y clasificar cada uno como “positivo”, “negativo” o “neutral” en sentimiento. O a partir de descripciones de actividades económicas en texto, asignarlas a sectores predefinidos (“Agricultura”, “Minería”, “Servicios”, etc.).

La categorización de texto normalmente requiere técnicas de procesamiento de lenguaje natural más avanzadas (palabras clave, algoritmos de clasificación), que escapan al alcance de este curso introductorio. Sin embargo, podemos mencionar casos *simples* donde *a partir de un texto determinamos una categoría explícita*. Un ejemplo sencillo: supongamos que tenemos una columna “Nivel de riesgo” con valores escritos como “bajo”, “medio” o “alto” en texto; podríamos mapear esos textos a categorías numéricas 1, 2, 3 (o etiquetas 0,1,2) que representen riesgo bajo, medio, alto respectivamente. Otro ejemplo: en registros de estudiantes podríamos tener la palabra “Aprobado” o “Reprobado” como resultado de un examen; es útil convertir eso a una categoría binaria (aprobado = 1, reprobado = 0 por ejemplo) para análisis cuantitativo.

En resumen, la idea aquí es transformar datos textuales en variables categóricas manejables. Una técnica común es crear reglas de búsqueda de palabras clave. Por ejemplo, si quisiéramos una categorización muy básica de tweets como “habla sobre transporte público” vs “no habla sobre transporte”, podríamos buscar palabras como “metro”, “micros”, “bus” en el texto; si aparecen, etiquetar ese tweet en la categoría “transporte”. Esta sería una forma rudimentaria de categorización de texto.

Como ejercicio sencillo, tomemos la columna de aprobados/reprobados mencionada y convirtámosla en 1/0 (que en el fondo es también codificar una variable categórica, tema que se solapa con el siguiente). Veamos cómo hacerlo en Python con una lista de ejemplo:

```
resultados = ["Aprobado", "Reprobado", "Aprobado", "Aprobado"]
codificado = [1 if r == "Aprobado" else 0 for r in resultados]
print(codificado) # [1, 0, 1, 1]
```

Aquí iteramos por la lista de strings y devolvemos 1 si el texto es "Aprobado" o 0 si no (asumiendo solo esas dos categorías posibles). Obtenemos [1, 0, 1, 1] en este caso. Esta simple conversión de texto a números nos permite, por ejemplo, calcular fácilmente la tasa de aprobación promedio con métodos numéricos (un promedio de [1,0,1,1] dará 0.75, o 75% aprobados).

Este ejemplo muestra una categorización binaria explícita. En casos de múltiples categorías de texto (como el ejemplo "bajo/medio/alto"), se podría hacer un mapeo similar con un diccionario: `mapping = {"bajo": 1, "medio": 2, "alto": 3}` y luego convertir cada texto a su código. Esto nos lleva al siguiente punto: cómo representar variables categóricas de forma adecuada.

Codificación de variables categóricas

Las variables categóricas son aquellas que representan categorías o etiquetas (ej. región, estado civil, tramo etario, color, etc.). En bruto pueden aparecer como texto ("Soltero", "Casado", ...), pero para muchos algoritmos necesitamos expresarlas numéricamente. Ya vimos un caso binario (Aprobado/Reprobado a 1/0). Generalizando, hay dos enfoques principales de codificación:

- **Label Encoding (codificación por etiqueta):** Asignar un número entero a cada categoría distinta. Por ejemplo, si tenemos colores {"Rojo", "Verde", "Azul"}, podríamos mapear Rojo->0, Verde->1, Azul->2. Esto convierte la categoría en un número único. Python/pandas puede hacerlo automáticamente (pandas tiene `pd.factorize` o `Categorical.codes`). Es simple, pero atención: ese número asignado no significa que exista un orden natural (a menos que la categoría lo tenga). En el ejemplo de colores, $2 > 1 > 0$ no tiene un significado matemático real, es solo un código.
- **One-Hot Encoding (codificación "uno-en-caliente"):** Crear columnas dummy para cada categoría, con valores 1 o 0 indicando la presencia de esa categoría. Siguiendo el ejemplo de colores, en lugar de una columna "Color" textual, crearíamos tres columnas: Color_Rojo, Color_Verde, Color_Azul. Si un registro era "Verde", entonces Color_Verde=1 y las otras 0. Este método evita introducir un orden arbitrario; todas las categorías se tratan por igual. El inconveniente es que aumenta las dimensiones (una columna por categoría).

Para variables categóricas *simples* (pocas categorías, no ordinales) es común usar one-hot encoding. Para variables ordinales (que **sí** tienen orden, ej. "Bajo < Medio < Alto") a veces un label encoding respetando ese orden (1,2,3) tiene sentido o se puede convertir a un número continuo si se cuantifica el nivel.

Veamos un ejemplo resultante de one-hot encoding con la lista de resultados aprobados/reprobados anterior. Si aplicáramos una codificación one-hot, tendríamos dos columnas: "Aprobado" y "Reprobado" (o equivalentes 1/0). Utilizando pandas para demostrar rápidamente:

```
import pandas as pd
res = pd.Series(["Aprobado", "Reprobado", "Aprobado"])
print(pd.get_dummies(res))
```

La salida sería una tabla como:

	Aprobado	Reprobado
0	1	0
1	0	1
2	1	0

Cada fila tiene un 1 en la columna de su categoría y 0 en la otra. Así el índice 0 y 2 (aprobados) tienen Aprobado=1, Reprobado=0; el índice 1 (reprobado) tiene Aprobado=0, Reprobado=1. En este caso, con dos categorías redundantes podríamos incluso guardar solo una columna (basta con una columna "Aprobado" con 1/0, donde 0 implica reprobado). Pero con múltiples categorías, el one-hot es útil.

En Python, `pd.get_dummies` es la forma rápida de hacer esto para un DataFrame completo. Por ejemplo, si tuviéramos una columna "Región" con valores ["Norte", "Centro", "Sur"], `get_dummies` crearía columnas `Region_Norte`, `Region_Centro`, `Region_Sur` con 1/0 indicando la pertenencia.

Resumen de esta sección: Transformamos y enriquecimos los datos realizando varias operaciones:

- **Normalización** de valores numéricos para estandarizar escalas.
- **Parseo de fechas** para obtener objetos fecha manejables.
- **Categorización de texto** cuando es necesario clasificar texto no estructurado en categorías.
- **Codificación de categóricos** para representar variables categóricas de forma numérica usable en algoritmos.

4. Agregación y combinación de datos

En muchas ocasiones, después de transformar los datos necesitamos obtener resúmenes agregados o combinar diferentes fuentes de datos para un análisis más completo. Esta sección cubre dos aspectos: (a) operaciones de agregación, como obtener estadísticas agrupadas (sumas, promedios, conteos por categoría), y (b) operaciones de combinación (unir datasets mediante un índice común, tipo `join/merge`).

Agregación de datos (estadísticas agrupadas)

Agregación significa sintetizar un conjunto de registros en valores resumen. Un ejemplo cotidiano: a partir de datos individuales de estudiantes (cada uno con su nota), podemos agregar por curso para obtener el promedio de notas por curso. O con datos de ventas diarias, agregamos por mes para ver ventas mensuales. La clave es usar alguna columna como clave de agrupamiento (ej. la categoría, la fecha, el grupo) y luego aplicar una función de agregación (suma, promedio, mínimo, máximo, conteo, etc.) sobre otra columna.

En Python, la librería `panda` simplifica mucho esto con operaciones como `groupby`. Ilustremos con un ejemplo simple: supongamos que tenemos un pequeño conjunto de datos de valores asociados a regiones de Chile, y queremos sumar los valores por región. Construyamos un DataFrame de ejemplo y apliquemos una agregación:

```
import pandas as pd

datos = {
    'Region': ['Norte', 'Norte', 'Centro', 'Sur'],
    'Valor': [10, 20, 15, 5]
}
df = pd.DataFrame(datos)
resumen = df.groupby('Region')['Valor'].sum().reset_index()
print(resumen)
```

Salida esperada:

	Region	Valor
0	Centro	15
1	Norte	30
2	Sur	5

Aquí df contiene 4 filas, con una columna Región (que puede ser por ejemplo "Norte", "Centro", "Sur") y otra de Valor numérico. Al agrupar por 'Region' y sumar 'Valor', obtenemos un nuevo dataframe resumen con una fila por cada región única: Centro total 15, Norte total 30 (porque Norte tenía 10+20), Sur total 5. Si en vez de sum() hubiésemos usado mean(), obtendríamos los promedios por región. Podemos agregar múltiples estadísticas a la vez (por ej. .agg({'Valor': ['mean','count']})) para obtener promedio y cuenta).

Para un contexto en Chile: podríamos interpretar que Valor son, por ejemplo, millones de pasajeros de transporte público, y Region indica macro-zona. Entonces el resultado nos diría cuántos millones de pasajeros suman en el Norte, Centro, Sur en total. Es un ejemplo inventado, pero sirve para ver el mecanismo. Otro ejemplo: si tuviéramos datos de inflación mensual por año, podríamos agrupar por año y obtener la inflación acumulada o promedio anual por año.

La agregación nos da insights condensados. En datasets grandes, mirar los datos sin agrupar puede ser abrumador; en cambio, las estadísticas agrupadas (por categoría, por periodo, por grupo demográfico, etc.) nos permiten entender tendencias generales e identificar patrones.

Combinación de datos (joins/merges)

En la práctica, los datos suelen venir en múltiples fuentes o tablas que hay que unir para obtener la imagen completa. Por ejemplo, podríamos tener una tabla con información económica por región y otra tabla con nombres y códigos de región; o una tabla de alumnos con sus notas y otra con sus datos personales. La combinación (también conocida como merge/join) enlaza dos conjuntos de datos por una o más claves comunes. Esto es análogo a las operaciones JOIN en SQL para bases de datos.

Hay distintos tipos de join:

- **Inner join:** conserva solo los registros que tienen match en ambas tablas (intersección).
- **Left join:** conserva todos los registros de la tabla izquierda añadiendo info de la derecha cuando hay match (si no hay match, queda nulo en esos campos).
- **Right join y Full join** son análogos (toda la derecha, o todos de ambos con nulos donde falte match, respectivamente).

Para este curso, enfocémonos en el caso típico de un **inner join** simple usando pandas. Construyamos dos DataFrames de ejemplo y combinémoslos:

```
df_ciudad = pd.DataFrame({
    'CiudadID': [1, 2, 3],
    'Poblacion': [1600000, 300000, 220000]
})
df_info = pd.DataFrame({
    'CiudadID': [1, 2, 3],
    'NombreCiudad': ['Santiago', 'Valparaíso', 'Concepción']
})
df_merged = pd.merge(df_ciudad, df_info, on='CiudadID', how='inner')
print(df_merged)
```


Salida:

	CiudadID	Poblacion	NombreCiudad
0	1	1600000	Santiago
1	2	300000	Valparaíso
2	3	220000	Concepción

En este ejemplo, `df_ciudad` podría ser una tabla con población por ciudad (CiudadID 1: 1.600.000 hab, etc.), y `df_info` una tabla de nombres de ciudad por ID. Al hacer merge por la clave 'CiudadID', obtenemos un dataframe combinado que une cada población con el nombre de ciudad correspondiente. Es decir, para CiudadID 1, juntó 1600000 con "Santiago" en la misma fila, y así sucesivamente.

Hemos realizado un inner join (podríamos haber omitido `how='inner'` porque es el valor por defecto). Si hubiera alguna ciudad en `df_ciudad` que no estuviera en `df_info` (u viceversa), no aparecería en el resultado por ser inner join (si quisiéramos conservarlas, usaríamos left or right join según el caso).

Contexto: Pensemos que `df_ciudad` viene del INE con poblaciones, y `df_info` es una lista oficial de códigos de ciudades con sus nombres. La combinación nos permite tener un cuadro final con nombre de ciudad y población juntitos. Este proceso es fundamental cuando nuestros datos están *normalizados* en varias tablas: por ejemplo, en una base de datos relacional, información complementaria se almacena en distintas tablas para no repetir datos. Al exportarlas y querer analizarlas, primero debemos *reunir* todo mediante merges.

Otra aplicación común: consolidación de datos de diferentes fuentes. Por ejemplo, ¡supongamos que obtuvimos un dataset de uso de tarjeta BIP! por comuna, y otro dataset independiente con población por comuna; podríamos hacer un join por comuna para calcular, por decir, viajes per cápita. Siempre que haya un campo en común (el nombre de la comuna, idealmente normalizado idéntico en ortografía), podemos juntarlos.

Operaciones de agregación y combinación en conjunto: Muchas veces ambos van de la mano. Imaginen: tenemos datos de escolares por colegio, y datos de colegios por región. Podemos agregar los escolares por región (agregación), pero para eso primero quizás tengamos que unir la tabla de alumnos con la tabla de colegios (join por código de colegio para traer la región de cada alumno, luego groupby región). Estas operaciones permiten integrar y resumir información compleja de manera relativamente simple con las funciones adecuadas.

Con esto, hemos aprendido a *filtrar, limpiar, transformar, agregar y combinar* datos, cubriendo todo el pipeline típico de preparación de datos para análisis. En el código, hemos usado estructuras y funciones tanto nativas (listas, bucles) como especializadas (pandas) para automatizar estos procesos, tal como indican los resultados de aprendizaje esperados (por ejemplo, el indicador 1.2.2 espera que implementemos en un lenguaje de programación estas operaciones).

5. Programación estructurada y modular

Hasta ahora hemos escrito fragmentos de código para tareas específicas (eliminar outliers, imputar valores, agrupar datos, etc.). A medida que construimos algoritmos más largos y complejos, es fundamental mantener un orden y claridad en el código, aplicando principios de *programación estructurada y modular*. Esto garantiza que nuestro programa sea legible, fácil de depurar, reutilizable y escalable.

¿Qué significa programación estructurada? En esencia, se refiere a escribir el código con una estructura lógica clara, evitando el “spaghetti code”. Implica usar apropiadamente las estructuras de control (if/else, bucles) en bloques bien definidos, evitar saltos de flujo impredecibles (como goto en lenguajes que lo tienen, que Python por suerte no), e indentación consistente. Python naturalmente nos fuerza a indentar y estructurar con bloques, lo cual ya fomenta esto. Pero más allá de la sintaxis, debemos pensar en la organización por funciones.

Modularidad: Consiste en dividir el problema en sub-problemas y soluciones parciales, implementando cada una en **funciones** o módulos independientes que luego se integran. Por ejemplo, en nuestro pipeline de datos podríamos tener una función `limpiar_datos(df)` que se encargue de outliers, missing, etc., otra función `transformar_datos(df)` que aplique normalizaciones y codificaciones, etc. Cada función realiza una tarea concreta. Esto aporta varios beneficios:

- **Reutilización:** Si mañana tenemos otro dataset que necesita la misma limpieza, podemos volver a usar `limpiar_datos` en lugar de reescribir todo.
- **Legibilidad:** El código principal se vuelve más legible: en vez de 50 líneas seguidas de transformaciones, vemos llamadas a funciones con nombres descriptivos que resumen cada paso.
- **Mantenibilidad:** Si hay un bug en la limpieza, vamos directo a la función `limpiar_datos`. Si queremos mejorar la transformación, podemos modificar `transformar_datos` sin tocar el resto, siempre que la interfaz (entradas/salidas) se mantenga igual.

Buenas prácticas de estilo: Junto con modularizar, hay una serie de convenciones (como la guía PEP8 en Python) que ayudan a la legibilidad:

- Nombres descriptivos para variables y funciones. Por ejemplo, `promedio_notas` es mejor que `pn` o `x`. `calcular_IMC(peso, altura)` es claro sobre lo que hace.
- Comentarios y/o docstrings: Explicar las partes complejas. Cada función debería idealmente tener un docstring breve que diga qué hace, sus parámetros y resultado. Ej:

```
def calcular_imc(peso, estatura):  
    """Calcula el Índice de Masa Corporal dado peso (kg) y estatura (m)."""  
    return peso / (estatura**2)
```

Esto documenta la intención para cualquier lector (o para uno mismo meses después).

- Formato consistente: indentación de 4 espacios en Python, líneas no demasiado largas, espacios después de comas, etc. Por ejemplo, `if x==5:` es menos legible que `if x == 5:`; pequeño detalle, pero en conjunto suman para un código limpio.
- Evitar repetición de código: Si encontramos que copiamos y pegamos un bloque, probablemente debamos meterlo en una función. El principio DRY (Don't Repeat Yourself) sugiere factorizar lógica redundante.

- Estructura general: A veces conviene escribir un *script principal* que llama a sub-funciones. Por ejemplo, al final del cuaderno podríamos tener:

```
def main():
    datos = cargar_datos("archivo.csv")
    datos = limpiar_datos(datos)
    datos = transformar_datos(datos)
    resultado = analizar_datos(datos)
    guardar_resultados(resultado)

if __name__ == "__main__":
    main()
```

Esto es una estructura típica de script, donde main orquesta todo. En un cuaderno interactivo quizás no definimos main, pero igual mantenemos secciones ordenadas.

Veamos un pequeño ejemplo de modularización: supongamos que repetidas veces vamos a normalizar distintas listas de valores numéricos, como hicimos antes. En lugar de reescribir el cálculo min-max cada vez, definamos una función reutilizable:

```
def normalizar_lista(valores):
    """Escala una lista de valores numéricos al rango [0,1]."""
    min_v = min(valores)
    max_v = max(valores)
    if max_v == min_v:
        # Si todos los valores son iguales, devolver todos 0 (o 1, todos iguales da igual)
        return [0 for x in valores]
    return [(x - min_v) / (max_v - min_v) for x in valores]

# Usando la función con diferentes listas:
print([round(x, 2) for x in normalizar_lista([50, 80, 90, 100])]) # [0.0, 0.6, 0.8, 1.0]
print([round(x, 2) for x in normalizar_lista([0, 5, 10, 15])])    # [0.0, 0.33, 0.67, 1.0]
```

Analicemos lo anterior:

- Definimos `normalizar_lista` con un docstring que explica su propósito.
- Agregamos un manejo especial: si `min == max` (es decir, todos los elementos son iguales), entonces no podemos dividir por cero rango; en ese caso, devolvemos todos ceros (ya que todos los valores son idénticos, su normalización puede definirse como 0, o toda una lista de 1, pero la elección de 0 es arbitraria aquí).
- Luego probamos la función con dos listas distintas. La función se reutiliza simplemente llamándola. Obtenemos en la primera `[0.0, 0.6, 0.8, 1.0]` (como vimos antes) y en la segunda `[0.0, 0.33, 0.67, 1.0]`.
- Si más adelante tenemos otra lista de valores que normalizar, simplemente llamamos `normalizar_lista(nueva_lista)`.

Notemos cómo mejora la claridad: nuestro flujo principal podría decir `datos['ingreso_norm'] = normalizar_lista(datos['ingreso'])` y entendemos inmediatamente que esa línea normaliza la columna de ingresos. Si no tuviéramos la función, veríamos la fórmula $(x - \min) / (\max - \min)$ repetida, pudiendo inducir error si la escribimos mal en algún lugar.

Otra ventaja de modularizar es que podemos probar cada función por separado (hacerle unit tests simples). Por ejemplo, podríamos verificar que `normalizar_lista([5,5,5])` devuelve `[0,0,0]` según definimos. Esto ayuda a validar partes del algoritmo (tema que trataremos más en la sección de validación).

Por último, mencionar buenas prácticas adicionales: usar control de versiones (Git) para colaborar en código, documentar cambios, y seguir estándares de estilo comunes al equipo. Si todos escriben con un estilo coherente, el código de un proyecto parecerá escrito por una sola persona, facilitando su lectura.

En resumen, esta sección enfatiza que no basta con que el código “funcione”; debe estar escrito de forma comprensible y ordenada. En contexto académico (y profesional), un código claro refleja un pensamiento claro. Además, la modularidad hace posibles proyectos más grandes, pues podemos dividir la carga entre funciones/módulos o entre miembros de un equipo.

Ahora que tenemos un código bien estructurado que limpia, transforma, agrega y combina datos, pensemos brevemente en los distintos tipos de datos que podemos enfrentar, porque lo que hemos visto aplica principalmente a datos tabulares (estructurados). ¿Qué pasa con datos no estructurados? Lo mencionamos parcialmente con el texto, pero lo abordaremos un poco más en la siguiente sección.

6. Introducción a datos estructurados vs. no estructurados

En el mundo de la ciencia de datos, no todos los datos lucen como una tabla de Excel. Datos estructurados son aquellos altamente organizados, generalmente en formato tabular (filas y columnas), bases de datos relacionales, CSVs, etc., donde cada registro tiene los mismos campos. Datos no estructurados, en cambio, no tienen un esquema fijo fácilmente reconocible – por ejemplo, el contenido de todos los tweets de Twitter, documentos de texto completos, imágenes, audio, JSON anidados, páginas web, etc. Existe también un término medio, datos semiestructurados, como JSON o XML, que aunque tienen estructura jerárquica, no encajan directamente en filas y columnas sin una transformación.

En este curso nos centramos en *datos tabulares estructurados*, ya que es el tipo más común para empezar (piensen en datasets clásicos: una tabla de clientes con columnas como RUT, nombre, edad, una tabla de ventas con fecha, producto, monto, etc.). Muchas de las técnicas que vimos (limpieza, agregación, merges) suponen datos estructurados en tablas o matrices.

Principales diferencias y consideraciones:

- **Formato y almacenamiento:** Datos estructurados viven en tablas, donde sabemos el tipo de cada columna (número, texto corto, fecha, etc.). Los no estructurados pueden ser, por ejemplo, un bloque de texto libre donde dentro puede haber cualquier cosa, o una imagen (matriz de píxeles) que no se interpreta sin un procesamiento especial. Almacenamos estructurados en bases de datos SQL, CSVs, DataFrames; los no estructurados pueden venir como archivos de texto, colecciones de documentos, etc.
- **Preprocesamiento distinto:** Muchas técnicas de preprocesamiento vistas aplican a tablas (outliers en columnas numéricas, valores faltantes en celdas). En datos no estructurados se requieren otras aproximaciones. Por ejemplo, en texto libre, la “limpieza” consiste en cosas como eliminar puntuación, pasar a minúsculas, remover palabras muy comunes (stopwords), etc. Para imágenes, limpiar podría implicar corrección de color, recortar bordes, normalizar tamaños. El concepto de *formato consistente* existe en no estructurados también: ej. en JSON asegurarse de que llaves y estructuras sean correctas, pero es diferente a un CSV.
- **Estructurado vs no estructurado en un mismo proyecto:** Muchas veces, proyectos de ciencia de datos combinan ambos. Ejemplo: supongamos que estamos analizando la satisfacción de usuarios de Metro de Santiago. Podemos tener datos estructurados (p.ej., una tabla con fecha, hora, número de pasajeros, retrasos, etc.) y datos no estructurados (como los textos de comentarios o reclamos de usuarios). Para un análisis completo, podríamos tener que procesar los textos (quizás

categorizarlos en temas o sentimientos) y luego unir esos resultados con los datos numéricos estructurados. Así obtenemos, por decir, una tabla final con indicadores numéricos y también una etiqueta de sentimiento promedio de comentarios en esa fecha, etc.

- **Herramientas:** Para estructurados, usamos SQL, pandas, etc. Para no estructurados, entran herramientas especializadas: por ejemplo, para texto (NLP) se usan librerías como NLTK, spaCy, etc., para imágenes bibliotecas de visión computacional como OpenCV, PIL, o frameworks de deep learning para procesarlas. El tratamiento es muy específico al tipo de dato.
- **JSON y semiestructurados:** Vale la pena mencionar JSON (JavaScript Object Notation) ya que es muy común al consumir APIs o almacenar documentos. Un JSON es básicamente texto estructurado en forma de *diccionarios/listas anidadas*. Por ejemplo, un JSON de una persona podría ser:

```
{
  "nombre": "Juan",
  "edad": 30,
  "aficiones": ["fútbol", "ajedrez"]
}
```

Esto tiene cierta estructura (llaves "nombre", "edad", etc.), pero no es tabular porque, por ejemplo, "aficiones" es una lista. Podemos considerarlo semiestructurado. Para transformarlo en tabla tendríamos que decidir, por ejemplo, tener una columna "aficion1", "aficion2" o guardarlo como una cadena. Pandas tiene métodos (`pd.json_normalize`) para aplanar JSON a tabular si es posible. En Python, podríamos cargar JSON con:

```
import json
persona = json.loads(json_str) # donde json_str es una cadena JSON
print(persona["nombre"]) # "Juan"
```

Esto nos da un diccionario Python que podemos manejar (estructura anidada de listas y dicts).

En la unidad presente, se discute principalmente *datos estructurados* y se "mencionan enfoques generales para textos, JSON, etc." (como decía el programa). Esto es lo que estamos haciendo: reconocer que existen otros tipos y que los principios de procesamiento se adaptan. Por ejemplo, hablamos de categorización de texto: eso es parte del procesamiento de un tipo no estructurado (texto) para convertirlo en estructurado (categoría). Otro ejemplo: si tuviéramos datos en imágenes, un paso de *transformación* podría ser extraer características numéricas de las imágenes (como promedio de color, o detectar objetos) para luego tener columnas numéricas en una tabla.

En resumen, datos estructurados vs no estructurados requieren enfoques distintos, pero comparten la filosofía de preprocesar, transformar y validar. En este curso nos enfocamos en tabulares, pero es bueno tener la perspectiva amplia:

- Trabajando con tablas (como DataFrames) cubrimos un amplio rango de aplicaciones (muchos problemas clásicos están en CSVs).
- Si mañana enfrentamos textos o JSON, sabremos que primero debemos convertirlos o extraer de ellos algo de estructura (palabras clave, campos, etc.) para luego aplicar técnicas similares de limpieza, agrupación, etc., en esa estructura derivada.

Con esto completamos la introducción. A continuación, retomaremos la idea de **validar** nuestro algoritmo de transformación de datos, aspecto crucial sin importar el tipo de dato o método usado.

7. Validación básica de algoritmos de transformación

Hemos diseñado y codificado algoritmos que limpian, transforman y combinan datos. Pero ¿cómo sabemos que el resultado es correcto? Aquí entra la validación básica: consiste en verificar que las salidas del proceso cumplen con lo esperado, tanto en términos de consistencia como de exactitud.

Validar en este contexto no es tan formal como la validación de un modelo predictivo, sino más bien hacer pruebas y comprobaciones lógicas sobre los datos transformados:

- **Revisar totales y conteos:** Un método simple es comparar alguna métrica agregada antes y después del procesamiento para asegurarse de no haber perdido o duplicado datos inesperadamente. Por ejemplo, si teníamos 1000 filas iniciales y no pretendíamos filtrar nada más que los outliers extremos (digamos se esperaban remover 5 filas), entonces el resultado debería tener 995 filas. Si aparecen 990, algo anda mal (quizás se eliminaron más de la cuenta). Igualmente, si unimos dos datasets, podemos verificar si el número de filas resultante coincide con lo previsto (en un inner join, debería ser como mucho el tamaño del más pequeño, etc.). Para conteos, pandas ofrece `df.shape` o `len(df)` para filas, pero también `value_counts` para categorías antes y después.
- **Verificación de sumas o promedios totales:** Si hacemos una agregación o combinación, un truco común es verificar que la suma total de un campo no cambió salvo lo esperado. Ejemplo: supongamos que dividimos un dataset en categorías y luego lo volvemos a juntar; la suma total de un campo numérico debería ser igual. O si concatenamos datasets (uno de 500 filas y otro de 300 filas), el combinado debe tener 800 (asumiendo no hay intersecciones removidas). Si estamos consolidando datos de ventas de varias sucursales, la suma de ventas consolidada debe igualar la suma de cada sucursal por separado.
- **Consistencia de valores:** Tras transformaciones, podemos comprobar condiciones lógicas. Por ejemplo, si calculamos una nueva columna "IMC" a partir de peso y altura, tal vez verificar que todos los IMC caen en un rango plausible (ej. $10 < \text{IMC} < 60$) y que no hay nulos inesperados. O después de normalizar una columna, verificar que efectivamente el mínimo es 0 y el máximo es 1 (podemos usar `min()` y `max()` sobre la columna normalizada).
- **Pruebas controladas (unitarias):** Cuando modularizamos por funciones, vale la pena probar cada función con inputs conocidos. Como hicimos al crear `normalizar_lista`, probamos listas sencillas donde sabíamos el resultado. Esto es una forma de *unit test* manual. Podríamos automatizarlo con `assert` en Python:

```
assert normalizar_lista([0, 5, 10]) == [0, 0.5, 1]
```

Si la aserción no se cumple, significa que la función no está produciendo lo esperado. En notebooks educativos, a veces no se usa `assert` pero sí imprimir resultados de ejemplo para inspección (lo cual es lo que hemos estado haciendo).

- **Validación cruzada con pequeños subsets:** Una técnica útil es correr el algoritmo de transformación en un subconjunto pequeño de datos (que podamos revisar manualmente) y comparar la salida con lo que obtendríamos si transformáramos esos datos a mano. Por ejemplo, tomar 5 filas de un dataset, aplicar nuestra función de limpieza, y verificar "a ojo" si efectivamente rellenó los nulos correctamente, eliminó los duplicados, etc. Si en esas 5 está todo bien, es más seguro (aunque no garantía total) que en el dataset completo también lo esté.
- **Datos de prueba (dummy):** En entornos profesionales, a veces se crean datos sintéticos que cubren casos extremos para probar el pipeline. Por ejemplo, incluir una fila deliberadamente duplicada, una con valor atípico extremo, una con un campo faltante, y ver si el algoritmo las trata como esperamos (quita duplicado, detecta outlier, rellena faltante, etc.). Esta es una buena práctica para asegurarse de que no se nos escapa nada.

Veamos un ejemplo concreto y sencillo de validación en código, continuando con algunas ideas de secciones previas. Supongamos que después de nuestra limpieza, eliminamos duplicados de una lista. Podemos validar que la cantidad de duplicados eliminados es correcta comprobando la longitud antes y después:

```
nums = [1, 2, 2, 3]
# Eliminación manual de duplicados
unicos = []
for n in nums:
    if n not in unicos:
        unicos.append(n)

print(len(nums), len(unicos)) # Imprime 4 3, antes y después
if len(nums) - len(unicos) == 1:
    print("OK: Se eliminó exactamente un duplicado.")
```

En este caso `nums` tenía 4 elementos con un duplicado del "2". Después del proceso en `unicos` quedan 3 elementos únicos. Imprimimos las longitudes: "4 3". Luego la condición verifica que la diferencia es 1, lo cual se cumple, y muestra el mensaje "OK: Se eliminó exactamente un duplicado." Esto nos da confianza de que la operación de eliminar duplicados funcionó *como esperábamos en este caso*. Si la diferencia hubiese sido distinta de 1 (por ejemplo 0 o 2), sabríamos que algo falló (o no eliminó el duplicado, o eliminó de más).

Podríamos encadenar más validaciones: por ejemplo, agregar

```
assert len(unicos) == len(set(nums))
```

que comprueba que el resultado tiene la misma cantidad de elementos que si hubiéramos usado un conjunto (método esperado). O si validáramos la normalización:

```
norm = normalizar_lista([50, 80, 100])
assert min(norm) == 0 and max(norm) == 1
para confirmar que la función efectivamente dejó min=0, max=1.
```

En una transformación de datos real, también podemos hacer verificación visual: graficar distribuciones antes y después. Por ejemplo, antes de limpiar outliers quizás un histograma tenía una cola larga, y después de removerlos la distribución se ve más centrada. O hacer un plot de valores faltantes (heatmap de nulls) antes y después para ver si se rellenaron. Estas inspecciones gráficas también son formas de validación (aunque cualitativas).

La validación debe hacerse en distintos conjuntos de datos si es posible, para asegurar que nuestro algoritmo generaliza. Por ejemplo, si desarrollamos el pipeline con datos de 2021, conviene probarlo sin cambios en datos 2022 para ver si algo falla (quizás en 2022 aparece una nueva categoría no vista antes y nuestro código no la maneja, etc.). Esto responde a lo que dice el indicador 1.2.4: probar la eficacia en distintos conjuntos, incluyendo datos estructurados y no estructurados.

Un caso interesante: supongamos que validamos nuestro pipeline tabular con un dataset de ejemplo (estructurado), luego intentamos aplicarlo a un JSON (semi-estructurado) transformándolo primero a tabla – podríamos descubrir que nuestra función de limpieza falla porque espera ciertas columnas fijas. Esa es una señal de que habría que hacer el algoritmo más robusto o modificarlo para ese caso, o simplemente documentar que funciona con cierto formato de entrada.

En conclusión, la validación básica es un paso indispensable tras la transformación: nos asegura que *"lo que programamos, hace realmente lo que creemos"*. En proyectos de datos, muchas veces detectamos

problemas no durante la codificación, sino al revisar los resultados y notar discrepancias. Por eso, siempre reserva tiempo para revisar y validar. Un algoritmo de transformación que pasa estas pruebas de verificación nos da confianza para ahora sí usar los datos resultantes en análisis o modelamiento sabiendo que reflejan correctamente las intenciones de limpieza y transformación iniciales.

8. Consideraciones de eficiencia en procesamiento de datos

El último tema que abordaremos es la eficiencia. Una vez que tenemos un algoritmo correcto y validado, debemos pensar en cuán bien escala en términos de tiempo de ejecución y uso de recursos (memoria principalmente). Esto es especialmente relevante cuando los conjuntos de datos crecen (hoy procesamos 1.000 filas, pero ¿y si mañana son 1.000.000?). Las consideraciones de eficiencia corresponden a asegurarnos de que nuestro código puede manejar volúmenes de datos moderados en Python sin volverse extremadamente lento o consumir toda la memoria, relacionándose con el indicador 1.2.3 (algoritmos eficientes) e introduciendo nociones básicas de complejidad (como menciona el programa).

Complejidad computacional (nociones iniciales): En ciencias de la computación usamos la notación Big-O para describir cómo crece el tiempo de ejecución de un algoritmo respecto al tamaño de la entrada n . Por ejemplo:

- Un algoritmo **$O(n)$** tiene tiempo lineal: duplicar la cantidad de datos duplicará el tiempo, aproximadamente.
- **$O(n^2)$** es cuadrático: si duplicamos n , el tiempo se cuadruplica (porque procesar cada par de elementos, por ejemplo).
- **$O(\log n)$** crece sub-lineal, muy eficiente en escala.
- **$O(n \log n)$** como la ordenación típica (sorting).
- **$O(2^n)$** o **$O(n!)$** son exponenciales o factoriales, intratables para n medianos (crecen demasiado rápido).

Sin entrar en formalidades, al diseñar algoritmos de datos debemos *identificar operaciones costosas*. Por ejemplo, bucles anidados sobre la misma lista de tamaño n implican $\sim n \cdot n = n^2$ operaciones. Si $n=1.000$, $n^2 = 1.000.000$ (un millón de pasos). Si n crece a 100.000, n^2 sería 10^{10} (diez mil millones de pasos) y probablemente impracticable en un script Python puro.

Veamos casos comunes en procesamiento de datos:

- Recorrer una lista con un solo loop es $O(n)$ – eso suele estar bien para n bastante grande (cientos de miles o millones pueden ser manejables en Python, aunque millones ya pueden sentirse lentos dependiendo de la operación en cada loop).
- Anidar dos loops para comparar cada par de elementos es $O(n^2)$ – esto se vuelve lento muy pronto. Por ejemplo, comparar cada registro con cada otro para buscar duplicados es innecesario cuando podemos usar estructuras como conjuntos o diccionarios que hacen búsquedas mucho más rápidas.
- Operaciones *vectorizadas* de librerías como NumPy o pandas internamente están en C y optimizadas, por lo que aunque conceptualmente hagan muchos cálculos, están cerca de la eficiencia de bajo nivel.

Optimización con estructuras nativas vs librerías: Python ofrece estructuras de datos que ayudan a mejorar la eficiencia:

- Usar un **conjunto (set)** o **diccionario** para búsquedas es mucho más eficiente que una lista. La operación x in lista es $O(n)$ (tiene que chequear potencialmente cada elemento hasta encontrarlo), mientras que x in set o dict es promedio $O(1)$ – básicamente constante, gracias a las tablas hash internas. Ejemplo: Si quisiéramos saber si un valor ya apareció antes (como hicimos en la

eliminación de duplicados manual), hacerlo con `if n not in unicos`: `unicos.append(n)` tiene que buscar en la lista cada vez (eso hace la solución de duplicados anterior $O(n^2)$ en el peor caso). En cambio, podríamos mantener un conjunto de vistos:

```
vistos = set()
unicos = []
for n in nums:
    if n not in vistos:
        unicos.append(n)
        vistos.add(n)
```

Aquí la verificación `n not in vistos` es $O(1)$ en promedio, así que el loop completo es $O(n)$. Esto marca una gran diferencia cuando `n` es grande.

- Usar pandas/numpy: Si podemos formular la operación en términos de vectores, la computación se realiza en C optimizado. Por ejemplo, sumar dos columnas de un DataFrame de 1 millón de filas con `df['C'] = df['A'] + df['B']` es muchísimo más rápido que iterar fila por fila en Python haciendo esa suma. En general, preferimos las funciones vectorizadas (`sum`, `mean`, `apply`, etc.) de pandas en lugar de loops `for` en Python puramente, siempre que sea posible.
- Evitar copias innecesarias de datos: Si tenemos un DataFrame grande y vamos a filtrar filas, usar métodos que no dupliquen los datos gigantes en memoria. Por ejemplo, `df = df[df['col'] > 0]` filtra en lugar de iterar y construir una nueva lista manualmente (pandas lo hace eficientemente en C). O en Python básico, a veces es mejor modificar una lista en el lugar que crear muchas nuevas listas temporales en bucles.

Operaciones particularmente costosas:

- Ordenamiento (sorting) es $O(n \log n)$. Para unos miles está bien, para millones puede tardar un poco pero es necesario a veces. Hay que ser consciente: ordenar 100 millones de números podría tardar bastante y consumir memoria, pero afortunadamente `pandas.sort_values` o `sorted()` en Python están bastante optimizados en C también.
- Joins/merges: Combinar datasets puede ser costoso dependiendo del tamaño, pero pandas usa algoritmos eficientes (hash join) que son aproximadamente $O(n + m)$ para unir `n` y `m` registros (lo cual es muy bueno). Sin embargo, si no se dispone de suficiente memoria, combinar dos tablas gigantes puede ser un problema.
- Aplicaciones fila a fila en pandas con `apply` o *loops implícitos* pueden ser lentas si la función Python que se aplica es compleja. A veces conviene lograr la operación con expresiones vectorizadas. Por ejemplo, si quisiéramos crear una columna categórica basada en otra, mejor usar `np.where` o indexing booleano en vez de un `apply` con una función Python pura.

Eficiencia espacial (memoria): No solo el tiempo importa, también la memoria. Trabajar con 100MB vs 10GB de datos es muy distinto. Algunas consideraciones:

- Si los datos caben en memoria, pandas puede manejarlos; si no, se necesitan estrategias como procesamiento por lotes, usar bases de datos, o librerías como Dask para computación out-of-core.
- En Python, cada objeto tiene cierto overhead; usar tipos primitivos (`int`, `float`) en arrays de NumPy es mucho más compacto que una lista de Python de `ints` (donde cada `int` es un objeto completo). Por eso, preferir arrays numpy/pandas cuando se pueda, en vez de grandes listas de Python, mejora memoria y velocidad.
- Eliminar referencias a objetos grandes cuando no se usan (ej. borrar DataFrames intermedios con `del` o reusar variables) puede ayudar al garbage collector a liberar memoria.

Ejemplo ilustrativo de eficiencia: Supongamos que necesitamos verificar si un elemento existe en una lista muy grande. Comparemos conceptualmente dos métodos:

1. Método ingenuo: recorrer la lista completa comparando uno por uno (peor caso, si el elemento no está o está al final, hacemos n comparaciones) – $O(n)$.
2. Método usando conjunto: convertir la lista a un set y luego hacer elemento in set – la conversión es $O(n)$ y la búsqueda es $O(1)$. Total $\sim O(n)$ pero con un factor constante mayor quizás (la creación del set). Si se van a hacer muchas búsquedas, la inversión de convertir a set se amortiza.
 - Aún mejor, si necesitamos muchas búsquedas, convertir a set una vez y luego múltiples in consultas es claramente ventajoso.

Para ver la diferencia de tiempo, podríamos usar `%timeit` en Jupyter o el módulo `time`. No ejecutaremos código de benchmarking aquí por brevedad, pero teóricamente:

- Con $n = 1e6$, búsqueda en lista tarda proporcional a $1e6$ pasos.
- Búsqueda en set tarda un tiempo casi constante (independiente de n) tras la creación del set.

Conclusión sobre eficiencia: En esta etapa inicial, no buscamos que optimices cada línea de código micro-optimizaciones, sino que desarrolles un olfato para cuando un enfoque puede ser demasiado lento y valga la pena usar otra estrategia. Algunas recomendaciones de eficiencia para el procesamiento de datos en Python:

- Evita bucles anidados pesados en Python puro; piensa si puedes usar una estructura de datos apropiada (set/dict) o vectorizar con numpy/pandas.
- Para tareas repetitivas sobre grandes datasets, investiga funciones ya implementadas (pandas tiene muchas utilidades en C).
- No obstante, *no sacrifiques claridad por eficiencia prematuramente*. Primero logra que el código funcione correctamente (y suficientemente rápido para los tamaños actuales). Si llegas a un cuello de botella, entonces optimiza esa parte específica. Por ejemplo, no conviertas todo tu código en una sola línea comprimida por ahorrarte 0.1 segundos, porque perderás legibilidad.
- Conoce los límites: Python, al no ser lenguaje de bajo nivel, tiene restricciones de velocidad para ciertos tamaños. Si algún día los datos son enormes (big data), se pueden usar otras herramientas (Spark, etc.), pero eso excede nuestro curso. Para volúmenes moderados (digamos hasta unos pocos millones de filas), con buen código Python/pandas se puede trabajar.

Con las consideraciones de eficiencia, cerramos los puntos conceptuales de la unidad. Vale la pena recapitular brevemente lo aprendido y cómo encaja con los resultados esperados de aprendizaje.

Preguntas de activación del aprendizaje:**COMPRUEBO MI APRENDIZAJE**

1. ¿Por qué es importante diseñar un algoritmo mediante pseudocódigo o diagrama de flujo antes de comenzar a programar en Python?
2. ¿Cuál es la diferencia entre normalizar y estandarizar datos numéricos, y en qué casos se aplicaría cada uno?
3. ¿Qué significa validar un algoritmo de transformación de datos y qué estrategias básicas pueden utilizarse para hacerlo?

En el siguiente QR puedes revisar las respuestas esperadas

**Información complementaria para el aprendizaje:****Información complementaria**

Link 1: una guía práctica sobre limpieza y transformación de datos en Python, con ejemplos de tratamiento de valores faltantes, duplicados, codificación y estandarización.

<https://www.geeksforgeeks.org/machine-learning/data-preprocessing-machine-learning-python/>

Link 2: Artículo introductorio que explica qué es pseudocódigo, cómo usar diagramas de flujo y cómo combinarlos para diseñar algoritmos antes de codificar.

<https://www.codecademy.com/article/pseudocode-and-flowchart-complete-beginners-guide>

Link 3: Tutorial actualizado que enseña cómo usar group by en pandas para agrupar datos, aplicar funciones de agregación y transformar los resultados.

<https://realpython.com/pandas-groupby>

Conclusión

En esta Unidad 2, “Algoritmos para Procesamiento y Transformación de Datos”, hemos recorrido el proceso completo de preparar datos mediante programación:

1. Comenzamos diseñando algoritmos con pseudocódigo y diagramas.
2. Implementamos técnicas de preprocesamiento, transformaciones (normalización, parseo, codificación), y operaciones de agregación y combinación en código Python.
3. Aplicamos principios de programación estructurada y modular, escribiendo funciones claras y código limpio.
4. Finalmente, validamos nuestras transformaciones con pruebas básicas y consideramos la eficacia y eficiencia en distintos escenarios, mencionando también datos no estructurados.

A lo largo de este cuaderno activador, se presentaron ejemplos prácticos (muchos con sabor local, desde inflación chilena hasta IMC y transporte) para ilustrar cada concepto de manera cercana. Cada sección se enlazó con la anterior para dar un hilo conductor: primero pensar, luego limpiar, luego transformar, luego resumir/unir, todo ello con buen estilo de código, y comprobando el resultado y rendimiento.

Con este conocimiento, estás en condiciones de programar algoritmos de procesamiento de datos de principio a fin sobre datasets reales. En la práctica, cuando enfrentes un nuevo proyecto de datos:

- Planifica tu enfoque (¿qué pasos de limpieza y transformación harás?),
- Implementa paso a paso probando en pequeños lotes,
- Estructura tu código en funciones reutilizables,
- Verifica que los resultados tengan sentido,

Referencia Bibliográfica

López, A., & Rojas, A. L. (2021). *Introducción a Python para estudiantes de ciencias*. Editorial de la Universidad Pedagógica y Tecnológica de Colombia (UPTC). <https://ipss.odilo.us/info/introduccion-a-python-para-estudiantes-de-ciencias-02608297>

Kelleher, J. D., & Tierney, B. (2021). *Ciencia de datos* (Serie de conocimientos esenciales de MIT Press). Ediciones UC. <https://ipss.odilo.us/info/ciencia-de-datos-la-serie-de-conocimientos-esenciales-de-mit-press-02574729>

Referencia del presente documento:

Instituto San Sebastián, Departamento Online (2025). *Apunte 2: Algoritmos para procesamiento y transformación de datos. Programación para la ciencia de datos*.